TU WIEN Informatics

Curriculum Learning in Partially Observable Markov Decision Processes

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Artificial Intelligence Techniques

by

Felix Dobrovich
Registration Number 12316656

to the Faculty of Informatics

at the TU Wien

Advisor: Univ.Prof. Dr. Thomas Lukasiewicz

Vienna, July 26, 2026

Felix Dobrovich	Thomas Lukasiewicz

Contents

Contents ii

1 Introduction 1

2 Related Work 3

3 Technical Background 5
3.1 Reinforcement Learning and MDPs 5
3.2 Solving RL-Problems 6
3.3 Curriculum Learning 8

4 Experiments 11
4.1 Introduction 11
4.2 The Key Environment 13
4.3 LockedRoom 15
4.4 Multiroom 17
4.5 Big Curriculum 19
4.6 Conclusion 24

5 Summary and Outlook 27
5.1 Summary 27
5.2 Outlook 28

A Hyperparameters 30

Overview of Generative AI Tools Used 31

List of Figures 32

Bibliography 33

CHAPTER 1

Introduction

Deep reinforcement learning (Deep RL) deals with complex optimization problems involving neural networks. It has proven to be highly effective and successful in several domains, often exceeding human performance. Whether in the context of strategy games such as chess and Go [8] or the fine-tuning of Large Language Models (LLMs) [6], deep reinforcement learning excels in a wide variety of tasks. It is a machine learning method that maximizes reward signals inside an environment. RL algorithms require an agent to collect data, which is used to update internal functions. The abstract formulation of these methods facilitates the application of reinforcement learning to a diverse range of problems. Combined with the capabilities of neural networks to approximate arbitrarily complex functions, many problems can be solved by a single, generalized algorithm [9].

Although theoretically applicable, practical limitations arise frequently. Neural networks require large amounts of training data to learn an underlying function. Similarly, reinforcement learning needs to generate a large number of samples to accurately approximate the optimal policy or value function within the environment. The combination of both techniques results in time-consuming training runs, where the effectiveness of sample generation is crucial [9]. Furthermore, it is challenging for agents to learn more general problem-solving skills, instead of overfitting to a specific task [3]. Additionally, vanilla reinforcement learning fails in difficult environments with sparse reward signals.

Curriculum learning is an extension to basic reinforcement learning that helps mitigate the previously mentioned problems. Instead of only training on one stationary task, a curriculum introduces multiple tasks of different difficulties. This approach was initially inspired by cognitive science [2], intuitively comparing the learning of a neural network to the learning of a child: starting with foundational concepts before progressively increasing task complexity. A natural follow-up question concerns the optimal strategy of ordering the tasks of a curriculum. One common method is to monitor the agent's learning progress to estimate the best task an agent should learn at the current training stage.

The monitor is commonly referred to as a teacher, hence the term Teacher-Student Curriculum Learning [5].

This work aims to demonstrate the benefits of such curriculum learning methods by empirically measuring sample efficiency, convergence, and generalization in chapter 4. Firstly, agents are trained on the same environment with different training methods. These include no curriculum, a random curriculum, and a learning-progress-weighted curriculum. Results suggest that a curriculum, even when not strictly necessary, can speed up convergence drastically. The prioritization of environments in which the agent is experiencing changes in return yields an even better ordering of tasks than randomly choosing environments for training. Secondly, this thesis demonstrates that curricula are able to solve complex MiniGrid environments, such as the LockedRoom and Multiroom environments, where traditional RL algorithms fail. Finally, the ability to scale and manage the learning of multiple complex environments is demonstrated by training on more than 15 environments concurrently. It shows that favoring learning progress becomes more essential as the curriculum grows in the number of tasks. Overall, it is shown that curricula are an excellent addition to RL training, enhancing generalization, convergence speed, and the ability to solve complex environments.

The thesis is structured as follows: Chapter 2 reviews the related work and historical context of curriculum learning. Chapter 3 details the technical background, covering the fundamentals of Markov Decision Processes (MDPs), Proximal Policy Optimization (PPO), and Teacher-Student Curriculum Learning (TSCL). Chapter 4 presents the experimental setup in the MiniGrid environment and evaluates the results across the different curriculum strategies. Finally, Chapter 5 concludes the work with a summary of the findings and an outlook on future research opportunities.

CHAPTER 2

Related Work

The field of Curriculum Learning is a highly promising one, as its conceptual foundations are not only intuitive but also fundamentally simple. The core idea of scaling task difficulty for learning algorithms goes back at least to Alan Turing, who famously proposed: "*Instead of trying to produce a programme to simulate the adult mind, why not rather try to produce one which simulates the child's? If this were then subjected to an appropriate course of education one would obtain the adult brain.*" [10]. While this theoretical vision is decades old, concrete algorithmic implementations of Curriculum Learning only emerged much later as the field of Machine Learning matured.

In a pioneering conceptual study, Jeffrey L. Elman [2] emphasized the importance of "starting small" by training a recurrent neural network on a word prediction task. By splitting the training data into simple and complex sentences and gradually exposing the network to the more difficult instances, Elman reported that the model was able to avoid poor local optima, a milestone that naive training on the entire dataset simultaneously failed to accomplish. Bengio et al. [1] were the first to propose a formal definition of what a Curriculum is. They formally described a Curriculum as a dynamic reweighting of the original training distribution over time, where difficult samples become more likely as training continues. They also conducted an experiment on shape classification. The dataset was again split into two partitions, where the easy partition consisted only of squares, circles and equilateral triangles with low variability in shape, position, size, rotation and color. A simple multilayer perceptron (MLP) was trained via stochastic gradient descent, minimizing the cross entropy loss. The additional hyperparameter introduced was "switch epoch", regulating the number of episodes trained on the easy-dataset before switching to the harder ones. The results showed a clear trend: increasing the 'switch epoch' parameter, and thereby extending the initial phase of training on easy examples, consistently led to better generalization and lower final error rates compared to the baseline without a curriculum [1].

Evidence suggests that curricula can enhance the learning performance of existing machine learning algorithms, however, explicitly labeling data does not scale well. Furthermore, previous definitions are not directly applicable to the domain of reinforcement learning, where difficulty dynamically increases or decreases as training continues. These problems as well as the benefits of CL in solving problems with sparse rewards motivated Matiisen et al. [5] to propose a new framework called "teacher student curriculum Learning" (TSCL). Conceptually, TSCL splits the underlying problem into a monitoring teacher. The teacher estimates the slope of the students learning curve for each task. One of the algorithms that were used for demonstrating TSCL used a window of fixed size to estimate the scores of the agent. These data-points were used to fit a linear regression model, where its regression coefficient estimates the slope of the actual learning curve. Matiisen et al. [5] demonstrated the effectiveness of their algorithms over hand-crafted or no curricula in different environments such as addition tasks and Minecraft.

Building on the concept of tracking learning progress, Kanitscheider et al. [3] scaled automated curriculum learning to a highly complex set of 107 tasks in Minecraft. In this setting an agent needs to collect a specific item (=task) in a "Simon Says"-like manner. Some of the tasks depend on each other (collecting wood before crafting a tool) which inherently suggests curriculum learning. The authors stated that the learning progress, which is the absolute difference between two exponential moving averages, elegantly follows the agent's learning curve. The automated curriculum guided the agent to collect 82 items, vastly outperforming any baseline method.

CHAPTER 3

Technical Background

3.1 Reinforcement Learning and MDPs

Reinforcement Learning (RL) aims to solve a set of problems in which a decision maker interacts directly with an environment trying to maximize a numerical reward signal. Commonly, these problems are framed as a Markov Decision Process (MDP). It consists of an agent that decides on an action A_t based on the state S_t at timestep t and an environment producing a reward R_t. A_t, S_t, R_t are elements of the sets of actions, states and rewards and t denotes the current timestep. The agent-environment interaction is illustrated in Figure 3.2. The agent maps a state $S_t \in S$ to an action A_t at time step t. The MDP serves as an abstract and flexible framework for all of the Reinforcement Learning problems [9].

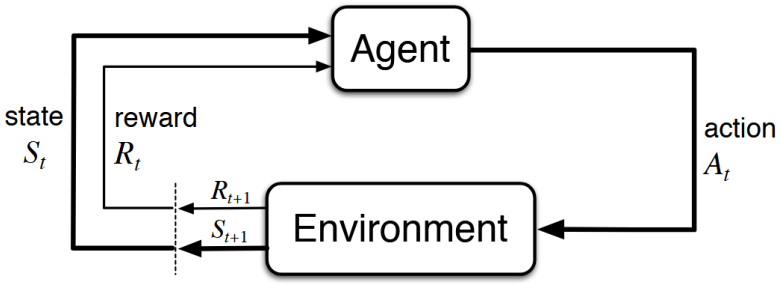


Figure 3.1: Agent-environment interaction-diagram in an MDP [9]

Furthermore, a finite MDP constrains the sets S, A, R to be finite in size. In this case the dynamics of the environment can be well described as discrete probability functions. The state transition function of a finite MDP is:

$$p(s', r|s, a) = \Pr\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\},$$

and it gives the probability of receiving reward r and being in state s' after taking action a in state s. The probabilities of p fully characterize the dynamics of the environment, so the next state only depends on the current state and action. The fact that the state S_{t-1} and action A_{t-1} are sufficient to predict the future of the process implies that all relevant information of the state (including relevant history) is encoded in every state and is called the Markov Property. While the exact dynamics of an environment are not important for this work, the constraint on the MDP being finite is [9].

Episodic tasks divide chains of (S_t, A_t, R_t)-tuples into episodes that end by reaching a terminal state (losing, winning, time-limit). The objective of an agent is to maximize the expected accumulated reward over an episode, which is called the return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{k+t+1}$, where γ is the discount factor for future rewards. Solving an RL-problem requires an agent to learn a policy function π that maps states to actions $\pi(s) \rightarrow a$ which is no worse than all other policies. This serves as the definition of optimal behavior. In other words, the agent needs to learn by interacting with the environment and change its behavior to accumulate more (maximum) return. In conclusion, the relevant MDPs for this work are finite and the tasks are episodic [9].

3.1.1 Partial Observability

The previously introduced framework of MDPs suffices for some RL problems. Most environments offer some sort of observation $o \in O$ where O is the observation space. In a fully observable MDP it holds that the state $o_t = s_t$, which makes choosing the optimal action a_t possible based on the observation. There is a real limit to the assumption of full observability, since many tasks are not fully described by a single instance of time (e.g. frame). Considering an example of a robot navigating inside a maze, acting only on its immediate observation will most likely fail. The observation does update the objective information, seeing a wall tells the agent where not to go, but can only be used if this information lies within the state. To address the issues of the Partially Observable MDP (POMDP) the agent needs to keep a belief state $b \in B \subset \mathbb{R}^{|S|-1}$ about the world. In theory this belief state carries the probabilities of every state. The agent acts on this belief state and updates it with the Bayesian update rule per observation. Since the belief space consists of all possible combinations of probabilities for all states, the described problem is now an MDP with state space B. A belief describes the dynamics of the new MDP. Rather than using a vector of $|S| - 1$ dimensions, this thesis will use a different approach to approximate the Markov Property using function approximation [4].

3.2 Solving RL-Problems

The objective an agent tries to maximize is the episodic return G_t. There are multiple algorithms that aim to find a policy $\pi(s)$ that maximizes said return. One important function that is widely used for learning is the value function. It estimates the future discounted rewards expected to be received when being in state S_t, thus it measures how "good" a specific state is. The value of a state entirely depends on the policy, which is

why the value function is defined as follows: $v_\pi(s) = \mathbb{E}[G_t|S_t = s]$, so the value of a state s under policy π is the expected return after starting in s and following π. Furthermore, the action-value function $q_\pi(s, a) = \mathbb{E}[G_t|S_t = s, A_t = a]$ gives the expected return of taking action a in state s then following π. Since action values implicitly describe a one-step lookahead, an optimal policy would simply result from taking the action in state s that maximizes the optimal action-value function. For MDPs with small, discrete state and action spaces, value functions can be represented in tabular form. However, this tabular approach is computationally intractable for continuous or high-dimensional spaces. This thesis uses function approximation to approximate the necessary functions via neural networks. The goal is then to find the parameters θ of those networks that approximate these optimal functions [9].

3.2.1 Proximal Policy Optimization (PPO)

A popular family of RL-algorithms directly optimizes their policies $\pi(a|s, \theta) = \Pr(A_t = a|S_t = s, \theta_t = \theta)$. The policy is a parameterized function which describes the probability of choosing action a in state s and uses this probability mass function to sample (S_t, A_t, R_t)-tuples. The objective of these policy gradient methods is to find the optimal parameters that maximize performance $J(\theta)$. In the episodic case, this term is equivalent to $v_{\pi_0}(s_0)$ where v_{π_0} is the true value function. In order to improve the policy, $\nabla J(\theta_t) = \nabla v_\pi$ (the policy gradient) is estimated for updating the weights. Normally, the gradient of the value of a state is complex and requires derivatives of the dynamics of the environment. The policy gradient theorem proves that the estimation of $\nabla v_\pi(s)$ does not need the dynamics of the process and simplifies to $\mathbb{E}_\pi[\sum_a q(S_t, a)\nabla\pi(a|S_t, \theta)]$ [9]. Expanding the term with the probability of the action A_t that was taken results in $\mathbb{E}_\pi\left[\sum_a q(S_t, a)\pi(a|S_t, \theta)\frac{\nabla\pi(a|S_t,\theta)}{\pi(a|S_t,\theta)}\right]$. When action A_t is actually selected, the term can be rewritten as $\mathbb{E}_\pi\left[q(S_t, A_t)\frac{\nabla\pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)}\right]$ and finally, since $q_\pi(s, a) = \mathbb{E}[G_t|S_t = s, A_t = a]$, it can be described as $\mathbb{E}_\pi\left[G_t\frac{\nabla\pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)}\right]$. The fraction of the term can be rewritten as $\nabla \ln \pi(A_t|S_t, \theta)$, which leaves a compact update rule scaled by the return: $G_t\nabla \ln \pi(A_t|S_t, \theta)$.

Often a baseline $b(s)$ is subtracted from the return to reduce variance resulting in a gradient of the form $(G_t - b(S_t))\nabla \ln \pi(A_t|S_t, \theta)$. The baseline $b(s)$ has to be independent of action a, and a common function used is the value function $v(s)$. It is an estimator for the advantage which is $A(s, a) = q(s, a) - v(s)$ and denotes how good an action is compared to the expected value of a state. This thesis uses an estimate of the advantage $\hat{A}_t = G_t - V(S_t)$ [9].

One downside of policy gradient methods is sample inefficiency, where sample-collection is the main cost factor for training RL-agents. This is because classical policy gradient methods are on-policy algorithms, meaning update samples have to be generated with the current policy. In the fraction of $\mathbb{E}_\pi\left[\hat{A}_t\frac{\nabla\pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)}\right]$ the denominator is collected by some old policy that would differ from update to update to the current one in the numerator. The correct update requires the old probabilities of the actions being selected to calculate

$\mathbb{E}_\pi \left[\hat{A}_t \frac{\nabla \pi(A_t|S_t,\theta_{\text{new}})}{\pi_{\text{old}}(A_t|S_t,\theta_{\text{old}})} \right]$. A common cause of failure in training models to represent the optimal and true functions of $v(s), \pi(s,a)$ is caused by unstable updates. Specifically, even small shifts in the probability ratio can cause the gradients to explode.

An excellent algorithm that leverages multiple updates on the same samples while keeping the updates stable is called Proximal Policy Optimization (PPO). This method aims to maximize the surrogate objective $\hat{\mathbb{E}}[r_t(\theta)\hat{A}_t]$ where $r_t(\theta) = \frac{\pi(A_t|S_t,\theta)}{\pi_{\text{old}}(A_t|S_t,\theta_{\text{old}})}$. PPO chooses $\min(r_t(\theta), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon))$ as the new factor multiplied by $\hat{A}_t$, giving the new objective of $\hat{\mathbb{E}} \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right]$. This prevents the importance sampling ratio from exploding and destroying updates in the training process. The chosen optimization algorithm for this work is PPO, as it provides a great balance of sample efficiency and stability [7].

3.3 Curriculum Learning

Although RL problems are solvable in theory, they often struggle in real-world scenarios. Oftentimes, they require a massive amount of data to learn meaningful behavior. This issue is exacerbated in environments with large state spaces and sparse reward signals (e.g., complex mazes or autonomous driving), which demand a highly specific sequence of actions before the agent receives any reinforcing feedback. In such sparse reward environments, the lack of directional gradient updates prevents the neural network from improving, causing the policy to remain highly stochastic with high entropy. For instance, reaching the goal state in a vast maze purely through random walk exploration is highly improbable.

A paradigm within Machine Learning called Curriculum Learning (CL) addresses this by controlling the order in which training samples are presented to the model. Intuitively, exposing a model to easier examples before gradually increasing the difficulty can significantly improve the learning process. Humans naturally learn through curricula; for example, mastering linear algebra builds upon the foundational concepts of addition and multiplication. An agent similarly struggles when the basics are not yet learned, raising the hypothesis that actively shaping the sequence of training examples can optimize Machine Learning tasks [1].

Bengio et al. [1] formalize a Curriculum: Let $P(z)$ be the training distribution, $W_\lambda(z)$ with $0 \leq W_\lambda(z) \leq 1$ and $W_1(z) = 1$ the weight assigned to training instance z at time step λ with $0 \leq \lambda \leq 1$. This results in a new training distribution $Q_\lambda(z) = P(z)W_\lambda(z)$ for time step λ. This essentially means that a Curriculum weights the samples by a factor at each timestep. It is also stated that the entropy and weights must be non-decreasing in λ, meaning the samples get "harder" and never diminish out of the training pool. Furthermore, Bengio et al. [1] state that the Curriculum makes the loss landscape more tractable. The loss function (criterion) $C(\theta)$ on a given task often suffers from being highly non-convex, making it prone to converging to a suboptimal local minimum. A curriculum has a family of such criteria $C_0(\theta), C_1(\theta), ...$ that increase in difficulty. $C_0(\theta)$

can be seen as a smooth version of subsequent $C(\theta)$ which are easier to optimize. The parameters θ can be pushed in the right direction ignoring the noise of difficult training samples and focusing on the core concepts of the problem. A curriculum can therefore be treated as a regularization method [1].

3.3.1 Teacher Student Curriculum Learning

A curriculum orders sub-tasks by their difficulty and switches at the right time to ensure maximum learning progression for the agent. This setting naturally requires a mechanism that decides when to switch and to which task. Consequently, the whole training process consists of two components with different objectives. Teacher Student Curriculum Learning (TSCL) is a field of Curriculum Learning that formalizes this process.

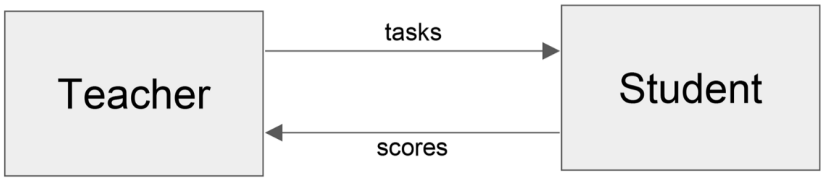


Figure 3.2: Student Teacher Framework [5]

At each timestep, the teacher decides on a task i from a set of N distinct tasks that the student has to perform. The teacher then processes the scores the student achieved, which are accounted for when picking the next task. The procedure describes a POMDP where state s_t represents the entire state of the student at time step t, including network parameters. An action a_t corresponds to the task selected by the teacher, who observes scores $o_t = x_t^{(i)}$, where $x_t^{(i)}$ is the score of task $i = a_t$. The reward r_t, which represents the rate of change of scores, is just $x_t^{(i)} - x_{t-1}^{(i)}$. One teacher episode supervises all episodes played by the student. Maximizing the teacher's return means maximizing

$$\sum_{i=1}^{N}\sum_{t=1}^{T}(x_t^{(i)}-x_{t-1}^{(i)}) = \sum_{i=1}^{N}(x_1^{(i)}-x_0^{(i)}+x_2^{(i)}-x_1^{(i)}+...+x_{T_i-1}^{(i)}-x_{T_i-2}^{(i)}+x_{T_i}^{(i)}-x_{T_i-1}^{(i)}) = \sum_{i=1}^{N}(x_{T_i}^{(i)}-x_0^{(i)})$$

with T_i being the last episode task i was trained on. Because of the telescoping sum, every $x_t^{(i)}$ but $-x_0^{(i)} + x_{T_i}^{(i)}$ cancels out. Assuming no knowledge of the agent, $x_0^{(i)} = 0$ means $\sum_{t=1}^{T} r_t = \sum_{i=1}^{N} x_{T_i}^{(i)}$. Consequently, maximizing the episodic return G_t of the teacher is the same as maximizing the final scores of each task i [5].

3.3.2 Learning Progress Algorithm

One way that a teacher chooses actions is by tracking progress. The teacher should switch to the task where the success probability changes the most in timestep t. This means

that for each task, the slope of the score $\Delta p_t^{(i)}$ needs to be estimated from a history of performance points. Kanitscheider et al. [3] proposed a method that computes two exponential moving averages (EMAs) where $p_t^{(i)} = (1-\alpha)p_{t-1}^{(i)} + \alpha x_t^{(i)}$ defines their updates. One EMA has a higher α than the other, which results in faster adaptation to new data. The absolute difference between p_{fast} (higher α) and p_{slow} marks the learning progress. The absolute difference is necessary to urge the teacher to select tasks with declining success rates to prevent catastrophic forgetting. These estimates can be converted into a probability mass function over all tasks, which can then be sampled. Kanitscheider et al. [3] used pre-processing before constructing the final probability distribution. For example, the raw p estimates were reweighted to magnify small success rates, increasing the likelihood of tasks at the beginning of their learning curve. Subsequently, the resulting learning progress values were standardized, transformed, and finally normalized into a probability function which focused 90% of its weight on the top 20% of tasks [3].

CHAPTER 4

Experiments

4.1 Introduction

This work demonstrates the benefits of Curriculum Learning through experiments using MiniGrid. MiniGrid is a simplistic world engine that enables the user to place walls and objects of different kinds in a 2D grid. Since grid sizes can vary, arbitrarily complex dungeon-like environments can be created despite their simplicity. The experiments use several of these environments from MiniGrid. They each vary in difficulty but have the same set of rules. A step penalty (-0.001) is given for each non-terminating state transition, and reaching the green goal square gives a reward of 1 and terminates the episode. Furthermore, the agent cannot see through walls and only has a vision of 7×7 tiles that are right in front of it. Consequently, the described decision process is only partially observable (POMDP). An agent following the optimal policy for every environment should reach the goal square as quickly as possible and terminate the episode.

4.1.1 Architecture

The reinforcement learning algorithm used to learn the environments is Proximal Policy Optimization (PPO) [7]. The learning network consists of a convolutional component which transforms the raw pixel input into a feature vector. Then a recurrent unit (GRU) transforms this input vector together with the hidden state into a new vector that acts as the state representation. The value and policy output heads are simple MLPs. The value head predicts a scalar; the policy head predicts logits for all 6 discrete actions. The recurrent unit is used because it may help the agent to remember picked-up items, seen objects or paths. Essentially, it tries to mitigate the problem of partial observability.

4.1.2 Training and Evaluation

The main training consists of the typical update cycle of PPO, updating the value and policy heads based on advantages. The training duration is measured in total samples throughout the training. For example, if the agent spends 100 transitions in environment a and 100 in b, the total cost for training is 200 transitions. This allows for a fair comparison since all the x-axes are comparable, unlike counting episodes that have different lengths.

In addition to measuring training duration, further metrics are used to evaluate training performance. The primary performance indicator is the success ratio SR, which essentially measures the ratio of successful episodes to total completed episodes over the last sample generation. An episode is considered completed if it has reached its time limit or the agent found the goal (success). Consequently, an SR aims to measure an average of how often an agent was able to terminate an episode before its time limit expires. This design choice was made because it has some advantages inside a curriculum over just the reward. Firstly, a reward is discounted over time, which results in different numeric values for a success for each environment and episode, depending on the number of steps an agent took. This makes comparing the progress between environments difficult as the rewards are inherently scaled differently. Secondly, for the following experiments small improvements in speed (completing environments faster) are not as important as completing the environments more frequently. This leads to the training shifting focus on learning more tasks instead of just optimizing the overall rewards of already learned tasks.

The learning progress LP_E of an environment/task E denotes the change in success ratios over training time. It is computed as $|FA_E - SA_E|$, where FA_E, SA_E are both moving averages updated via $MA_E^{(t)} = (1-\alpha)MA_E^{(t-1)} + \alpha SR_E^{(t)}$ with $\alpha_{FA} > \alpha_{SA}$. The LP can be used as a metric for the momentum of training inside an environment. Using the absolute difference between slow and fast average results in a high numeric LP both when success ratios go down or up, so in either case, a high LP urgently suggests to train on the environment.

The teacher needs to pick an environment for the next sample generation based on its collected LPs. In the following experiments, this is done by forming a simple probability mass function based on the environments' LPs $p(e) = \frac{LP_e}{LP_{total}}$ or a uniform distribution if $LP_{total} = 0$. This function can be sampled any time to get the teacher's suggestion for the next environment. Since every LP is initialized as 0, the teacher chooses randomly among the available environments with probability ϵ, ensuring that every task is explored over the course of training. This method will further be called LP and it aims to outperform randomly sampling tasks. Consequently, it will be compared against a method called Uniform (labeled as U in plots) that randomly selects environments. Another method called Baseline will directly train on the specific target environment, if one exists. For LP and Uniform, metrics like the teacher's estimated learning progress (Teacher LP), or the number of times an environment was selected, will be plotted besides the success ratios,

since they showcase the teacher's behavior effectively. For Baseline and Uniform these metrics do not exist, and only their success ratios will be plotted, mostly as a comparison against the Baseline in the same plot. These success ratios will be in blue for LP, pink for Uniform and gray for Baseline.

Strict determinism in the experiments significantly increases execution time; therefore, the following experiments introduce slight non-determinism. To compensate this, whenever possible, multiple runs were evaluated to gain statistical significance. The time steps are accumulated into bins of certain sizes of steps. The plots show the mean of those buckets and their standard deviation (shaded), resulting in a summary of training runs in one plot.

4.1.3 Goals

The training of reinforcement learning agents is known to be sample-inefficient. Millions of transitions are wasted because the agent's value function is yet far from the actual value function and updates are performed on trajectories without reward signals. Initially, an agent's policy has high entropy, meaning that initial action probabilities are distributed nearly uniformly. The following experiments aim to steer the policy into a more optimal parameter-space as quickly as possible, so that training on the target task becomes more sample-efficient. One goal of the experiments is to speed up convergence by using fewer samples for equal/better training results.

Many of the experiments fall within the category of sparse rewards. Essentially, the target environments the agent must learn are complex and require a specific sequence of actions to be executed to reach the goal. Policies are initialized randomly, making this very specific sequence of actions unlikely to be taken by the agent. This is because assuming a random policy, taking a specific action happens with a probability of $1/6$ (since in MiniGrid there are 6 actions). Under a highly stochastic policy, the chances of taking an extremely specific trajectory of actions over N steps has the slim probability $(1/6)^N$. Most of the time, more than just one path leads to the goal state but the ratio of successful paths to unsuccessful paths usually becomes smaller the larger the environment. In easier settings, the policy gets reinforced by frequent reward signals, making specific paths more likely to happen. Without those reward signals, a policy may waste a huge amount of samples due to the exponential behavior explained before, or not even learn anything. Since all environments only offer reward on the green goal square, many of the bigger ones automatically suffer from the problems of sparse rewards.

4.2 The Key Environment

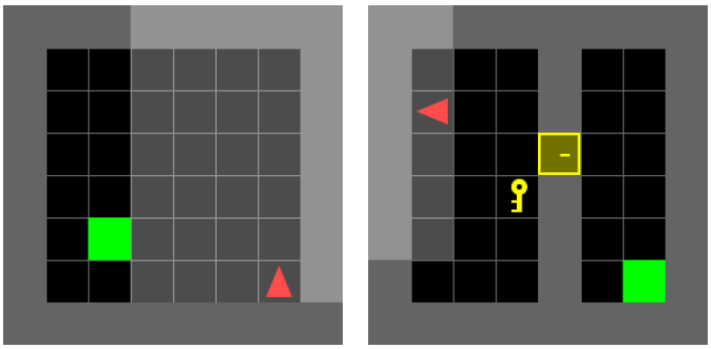
4.2.1 Problem

The Key-environment (Figure 4.1b) is one of the simpler environments provided by MiniGrid. To complete it, the agent needs to pick up the key, unlock the door and

reach the goal square. Although this environment usually does not require any sort of curriculum, it can be used to demonstrate sample efficiency.

4.2.2 Setup

The curriculum uses two tasks: Empty (Figure 4.1a) and the target task Key (Figure 4.1b). Empty is MiniGrid's easiest environment and can be optimized quickly. Despite its simplicity, it requires the agent to learn essential navigation and goal-reaching capabilities.



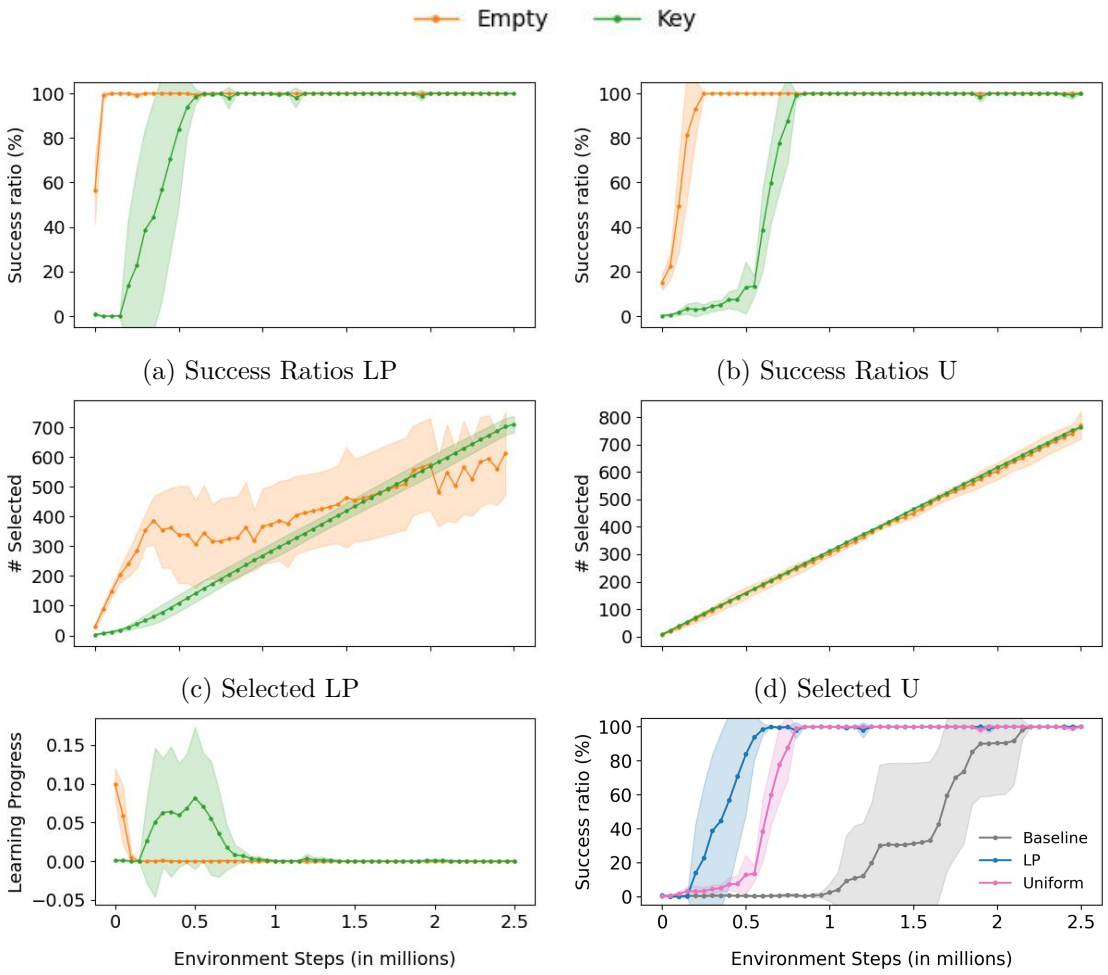
(a) Empty (b) Key

Figure 4.1: Curriculum for Key

4.2.3 Results

Baseline, Uniform and LP were tested with five different seeds and a bucket size of 50,000. Initially, the agent achieves a high success ratio in the Empty environment (Figure 4.2a). Baseline achieves its first success in Key after one million steps and converges after completing about two million steps (Subfigure 4.2f). In contrast, the LP-method first succeeds after 250,000 steps and converges at about 600,000 steps. Baseline requires more than twice the number of samples compared to LP to complete Key (Figure 4.2f). Uniform converges at roughly 750,000 timesteps, significantly outperforming Baseline. The teacher monitors high values for the LP in Empty (Figure 4.2e) resulting in almost 400 selections for the environment at 400,000 timesteps and less than 100 for Key (Figure 4.2c). Simultaneously, Uniform has chosen both environments 150 times (Figure 4.2d).

The prioritization of Empty by the teacher is the intuitive approach for ordering tasks in the curriculum, if it were designed by hand. As a result, LP requires about 20% fewer samples to solve Key (Figure 4.2f). Both LP and Uniform require less than half the amount of samples of Baseline and demonstrate the sample efficiency of a curriculum method over direct training on one task.



(a) Success Ratios LP

(b) Success Ratios U

(c) Selected LP

(d) Selected U

(e) Teacher LP

(f) Target SR

Figure 4.2: Key training

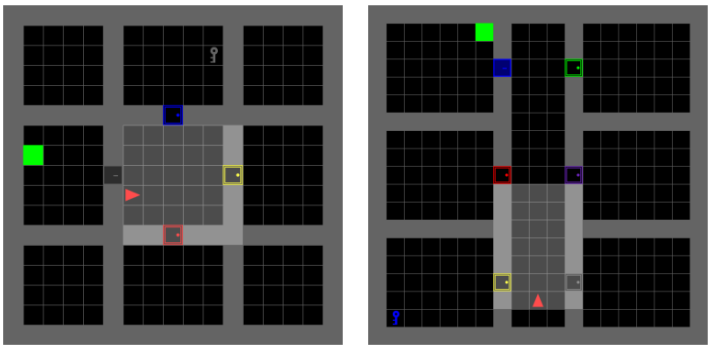
4.3 LockedRoom

4.3.1 Problem

The target of training is to learn a more complex MiniGrid environment called LockedRoom (Figure 4.3b). It consists of five unlocked rooms, one of which contains a key. The sixth room, which contains the goal square, is locked and can be unlocked by said item. The agent has to search for the key in the unlocked rooms to get access to the goal room. This sequence is unlikely to occur by randomly taking actions, and reward occurs only by reaching the goal, making this environment difficult to optimize due to sparse rewards.

4.3.2 Setup

The curriculum used splits this problem into several subtasks. In this case, four environments are introduced to train the agent to complete the target task. Empty (Figure 4.1a) is included for basic navigation and goal finding skills. Key (Figure 4.1b) requires the agent to open locked doors, which is also necessary to complete the target task. Color (Figure 4.3a) is just a simplified version of LockedRoom (Figure 4.3b). The baseline of the experiment uses no curriculum and trains only on the LockedRoom environment.



(a) Color (b) LockedRoom

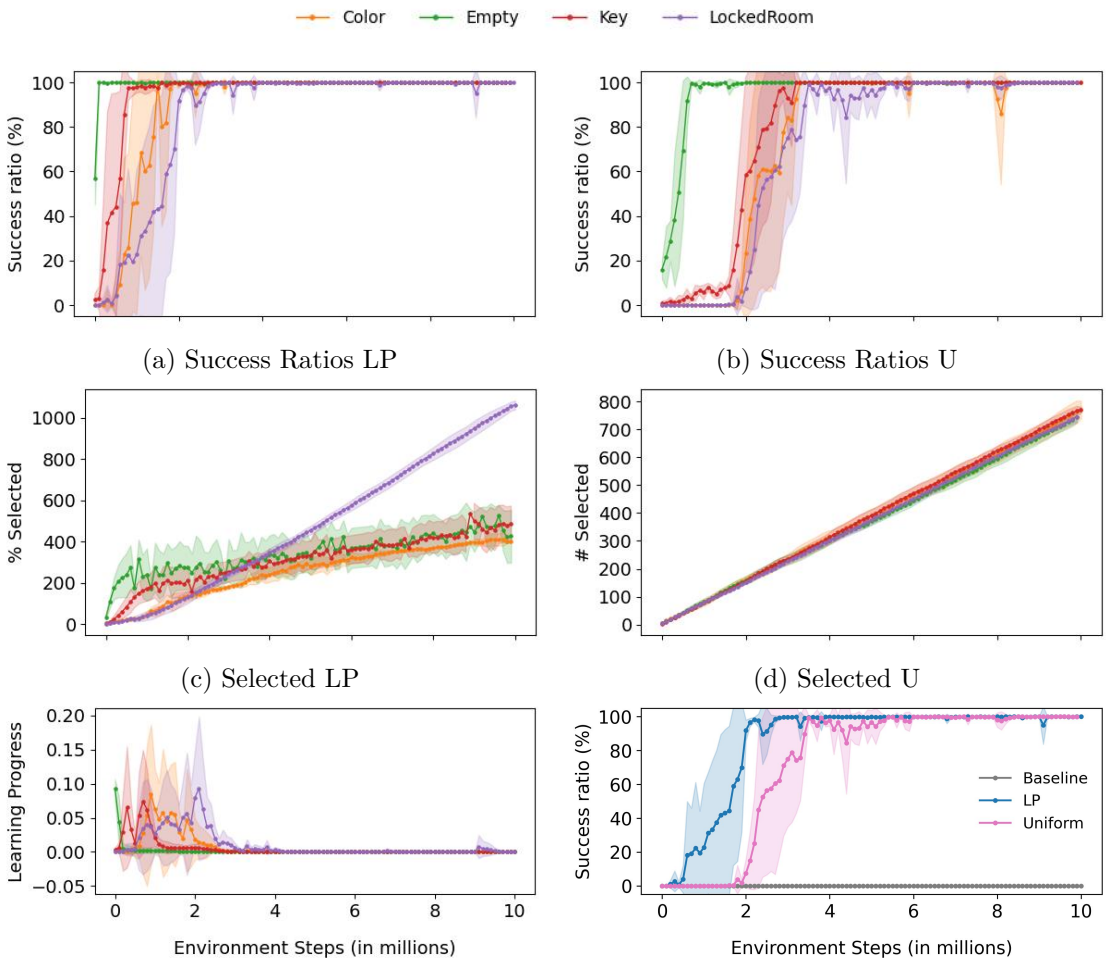
Figure 4.3: Curriculum for LockedRoom

4.3.3 Results

Baseline, Uniform and LP were tested with five different seeds and a bucket size of 100,000. Both LP and Uniform complete LockedRoom while the baseline completely fails. The Baseline does not achieve any reward after completing 10 million timesteps (Figure 4.4f). LP chooses almost exclusively Empty (Figure 4.4c) in the first 1 million steps. Concurrently, the teacher's estimated learning progress and the success ratios for Empty rise to their peaks (Figure 4.4a). At the 500,000 step mark, the same can be observed for Key, Color, and LockedRoom in this exact order. Simultaneously, initial success in reaching the goal state occurs for LP, followed by Uniform after 2 million steps. After around 3.5 million steps, LP reaches convergence, whereas the Uniform approach converges after 6 million steps.

In LP, the success ratios for Empty and Key show high values in the first million timesteps and low values of Color and LockedRoom (Figure 4.4a). Thus, higher deltas for the reward get monitored by the teacher (Figure 4.4e), which leads to the disproportional distribution of selections (Figure 4.4c). As a result, Empty and Key have already appeared 200 times in training in comparison to less than 50 times for Color and LockedRoom (Figure 4.4c). On the contrary, Uniform does not discriminate despite low deltas in reward (by definition) for LockedRoom and Color in early training (Figure 4.4b), selecting each environment roughly 75 times (Figure 4.4d). At the end of training, LP still chose LockedRoom more often than any other environment and more often than Uniform chose

LockedRoom (Figure 4.4d). This unbalanced usage of training resources by LP requires 40% fewer samples than equally sampling from the environments (Figure 4.4f).



(a) Success Ratios LP

(b) Success Ratios U

(c) Selected LP

(d) Selected U

(e) Teacher LP

(f) Target SR

Figure 4.4: LockedRoom training

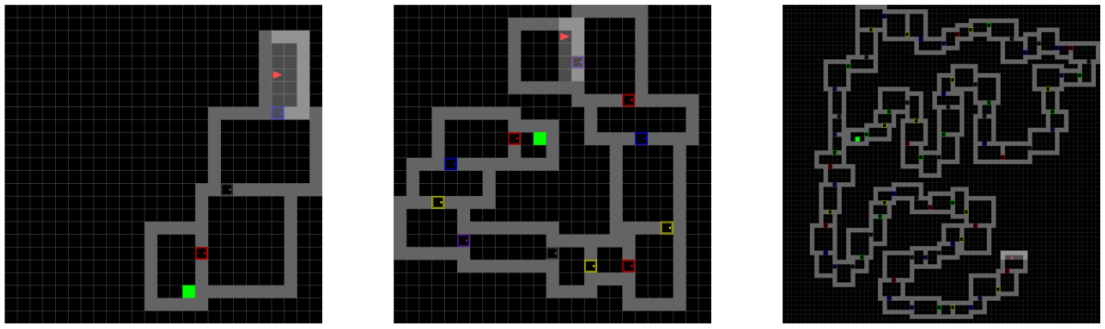
4.4 Multiroom

4.4.1 Problem

The Multiroom environment (Figure 4.5) consists of multiple rooms chained together. An agent must open all the doors to get to the goal state. Since adjustable, the difficulty of the environment depends solely on the number of rooms. As this number increases, the specific paths an agent must take become exponentially more unlikely assuming a random policy. Consequently, agents struggle in Multiroom with a large number of rooms due to sparse rewards.

4.4.2 Setup

This experiment compares Uniform against Baseline. The Baseline trains directly on Multiroom with 50 rooms, whereas Uniform randomly generates environments with 2-4 rooms. The aim is to spend minimal time on training and generalize to bigger sizes of Multiroom (Figure 4.5c). LP was not used in this experiment, since randomly sampling environments with 2-4 rooms converged conveniently and quickly anyway.

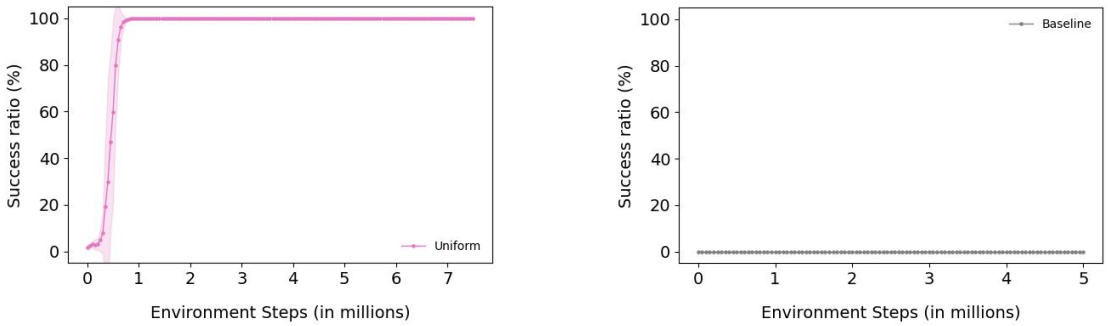


(a) 4 rooms (b) 12 rooms (c) 50 rooms

Figure 4.5: Multiroom

4.4.3 Results

Baseline and Uniform were tested with five different seeds and a bucket size of 50,000. Uniform converges on average at around 1 million timesteps (Figure 4.6a). Training the agent directly on the 50-rooms environment does not achieve any reward signal during training (Figure 4.6b) and learning does not happen. Despite only training on a maximum of 4 rooms, the resulting models from the Uniform training achieve an average success ratio greater than 99.5% in 1000 test runs on the 50 rooms Multiroom environment. The results show that optimizing the easy subproblem helps mitigate the problem of sparse rewards. In addition, identifying the sub-patterns (e.g opening the door, remembering the global direction) leads to strong generalization in bigger instances of the same environment.



(a) Multiroom with 2-4 rooms

(b) Multiroom with 50 rooms

Figure 4.6: Multiroom training

4.5 Big Curriculum

4.5.1 Problem

Thus far, experiments focused on learning a specific target environment. This one focuses on learning a set of tasks without declaring one as the most important one. A big curriculum is designed with structurally dependent tasks, all of which an agent ideally masters.

4.5.2 Setup

For bookkeeping reasons, the tasks will be split into logical groups, but the training happens on all environments at once. This grouping was done by hand, inspired by the environment's similarity or expected difficulty.

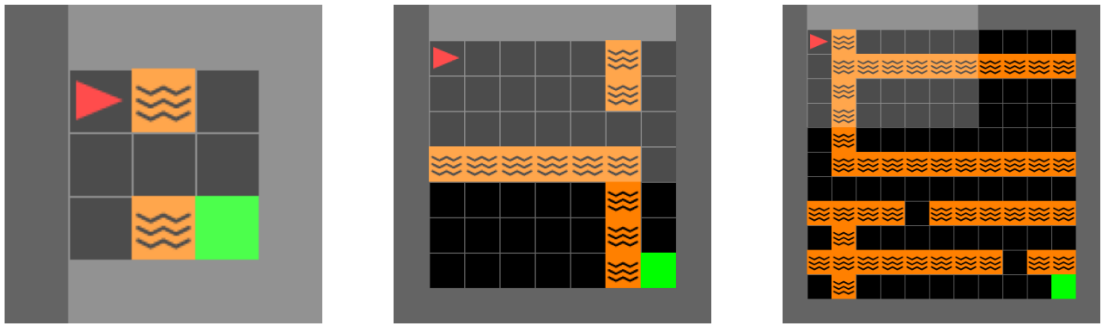
Basics

Environments of the group Basics have been introduced in former experiments. They include Empty (Figure 4.1a), Multiroom (Figure 4.5a) and Key (Figure 4.1b). Their contribution to the overall training is to establish default behaviors for the agent. Examples of this would be to always walk onto the goal square whenever it is seen (Empty), to pick up keys (Key) or to open doors (Multiroom). For further use in complex tasks, those default behaviors are best adopted in early stages of training, so that samples are used efficiently.

Lava

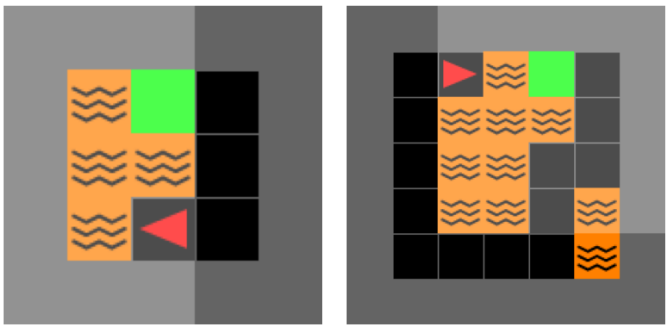
This group includes all the easier environments that have lava squares in them. The main goal is to learn Lava 2 (Figure 4.7c) and Lavamaze 1 (Figure 4.8b). Since the agent tends to struggle at tasks involving lava, these tasks are included with multiple difficulties. The enumeration number i for Lava i simply marks the difficulty of the

environment. A higher i corresponds to a bigger, more challenging environment. The Lava tasks (Figure 4.7) require the agent to cross large amounts of lava. Navigation skills through a maze obstructed with lava are taught through the Lavamaze environment.



(a) Lava 0 (b) Lava 1 (c) Lava 2

Figure 4.7: Lava



(a) Lavamaze 0 (b) Lavamaze 1

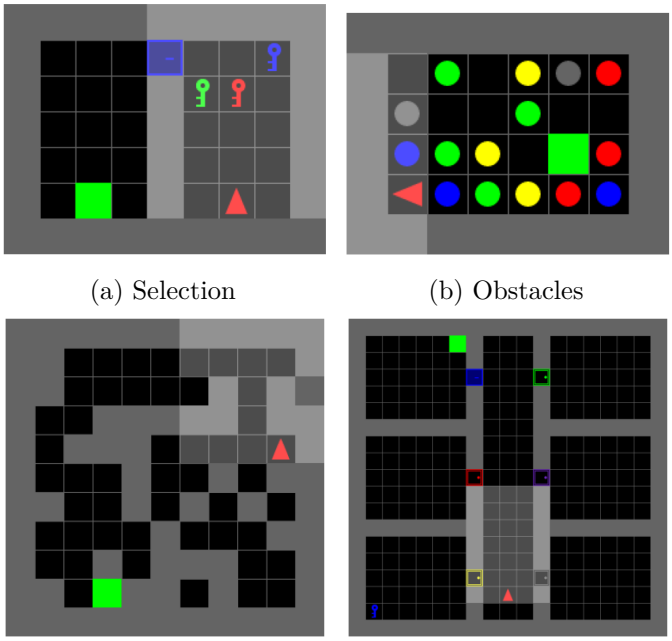
Figure 4.8: Lavamaze

Intermediate

This group contains more challenging environments than in the Basic one. They contain all its difficulties and expand them or introduce new hurdles. For instance, Selection (Figure 4.9a) requires an agent to distinguish between differently colored keys, pick only the fitting key, or drop a false one. Operating despite partial observability is trained through the Maze environment (Figure 4.9c). In Obstacles (Figure 4.9b), randomly placed balls block the path to the goal, forcing the agent to clear its way to reach the target state.

Combined

These types of environments are dependent on the skills learned before. They combine previous skills, introduce new ones, and are in general of larger scale. They are the most



(a) Selection (b) Obstacles

(c) Maze (d) LockedRoom

Figure 4.9: Intermediate

difficult environments of the entire training. They are simply enumerated from 1 to 5, where the index does not necessarily imply a strict hierarchy of difficulty.

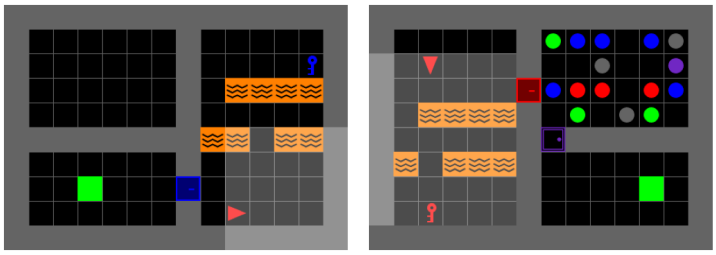
Combined 1 (Figure 4.10a): Key (Figure 4.1b) + Lava (Figure 4.7c).
Combined 2 (Figure 4.10b): Combined 1 + Obstacles (Figure 4.9b).
Combined 3 (Figure 4.10c): Lavamaze 1 (Figure 4.8b)+ Key.
Combined 4 (Figure 4.10d): Multiroom (Figure 4.6a) + Obstacles.
Combined 5 (Figure 4.10e): Combined 3 + LockedRoom (Figure 4.9d).

4.5.3 Results

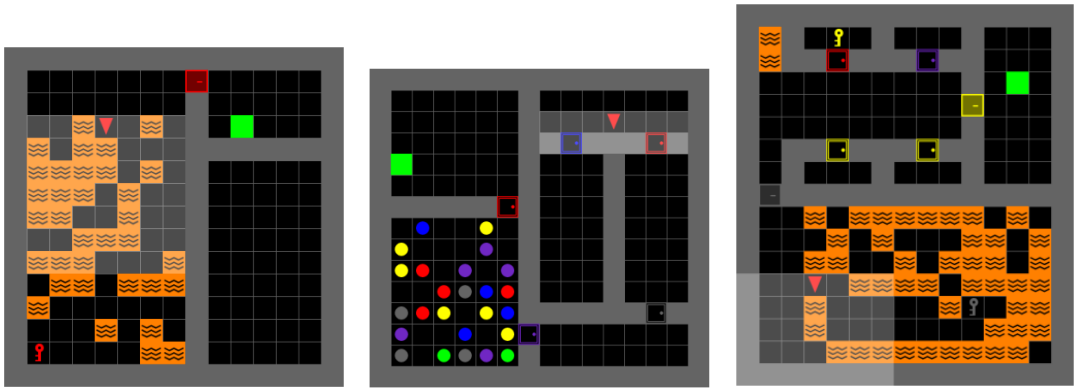
Due to extremely high variance, the bucket size was increased to 500,000 for readability reasons. 5 runs were conducted for Uniform and LP, each collecting 50,000,000 samples.

Basics

LP solved Empty (Figure 4.11d) and Key (Figure 4.11e) only slightly faster than Uniform. A slightly larger gap can be observed for Multiroom (Figure 4.11f), as LP reached a stable 100% success ratio at time step 10,000,000, in comparison to Uniform at 30,000,000. Overall, both methods completed these environments during training. Teacher metrics



(a) Combined 1 (b) Combined 2



(c) Combined 3 (d) Combined 4 (e) Combined 5

Figure 4.10: Combined

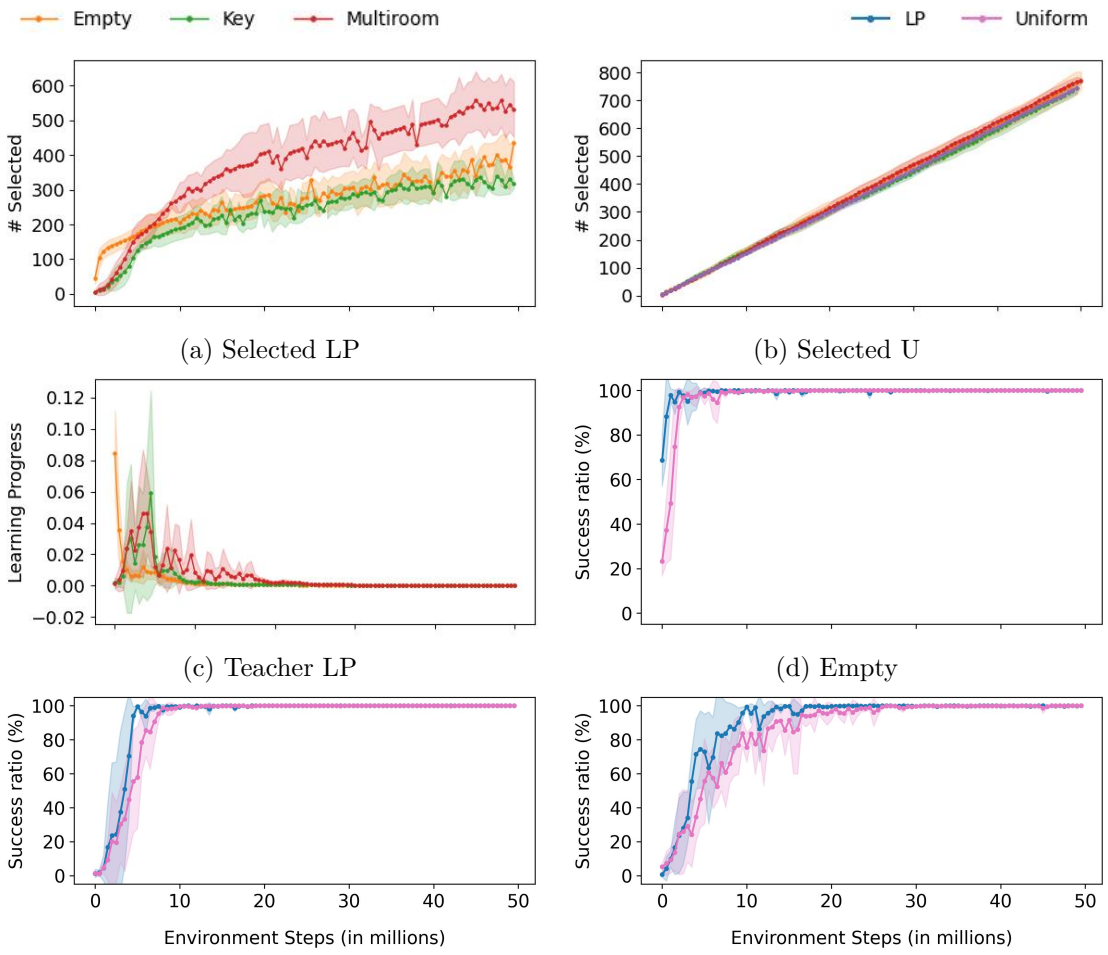
such as learning progress (Figure 4.11c) and selections (Figure 4.11a) show an early preference for Empty and Key, while Multiroom was overall selected more often.

Lava

Lavamaze 1 was solved in the first 30,000,000 timesteps by LP and in the first 40,000,000 by Uniform (Figure 4.12e). The success ratio plots for Lava 2 demonstrate the agent's struggle in lava-type environments, as announced in the setup section (Subsection 4.5.2). The agent does not fully converge robustly in both cases, and Uniform shows a slight edge in this regard (Figure 4.12d). The teacher metrics (Figure 4.12) as well show a nonzero learning progress over all timesteps, combined with the highest number of selections in the entire curriculum (Figure 4.12a). This indicates that either an increase or loss in success ratio is constantly perceived by the teacher. In conclusion, these crucial environments were learned sufficiently well.

Intermediate

Both methods succeeded in environments of this group. LP converges in the Selection environment (Figure 4.13d) over 15,000,000 timesteps earlier than Uniform. Apart from Selection, there is no clear outperformance noticeable (Figure 4.13).



(a) Selected LP

(b) Selected U

(c) Teacher LP

(d) Empty

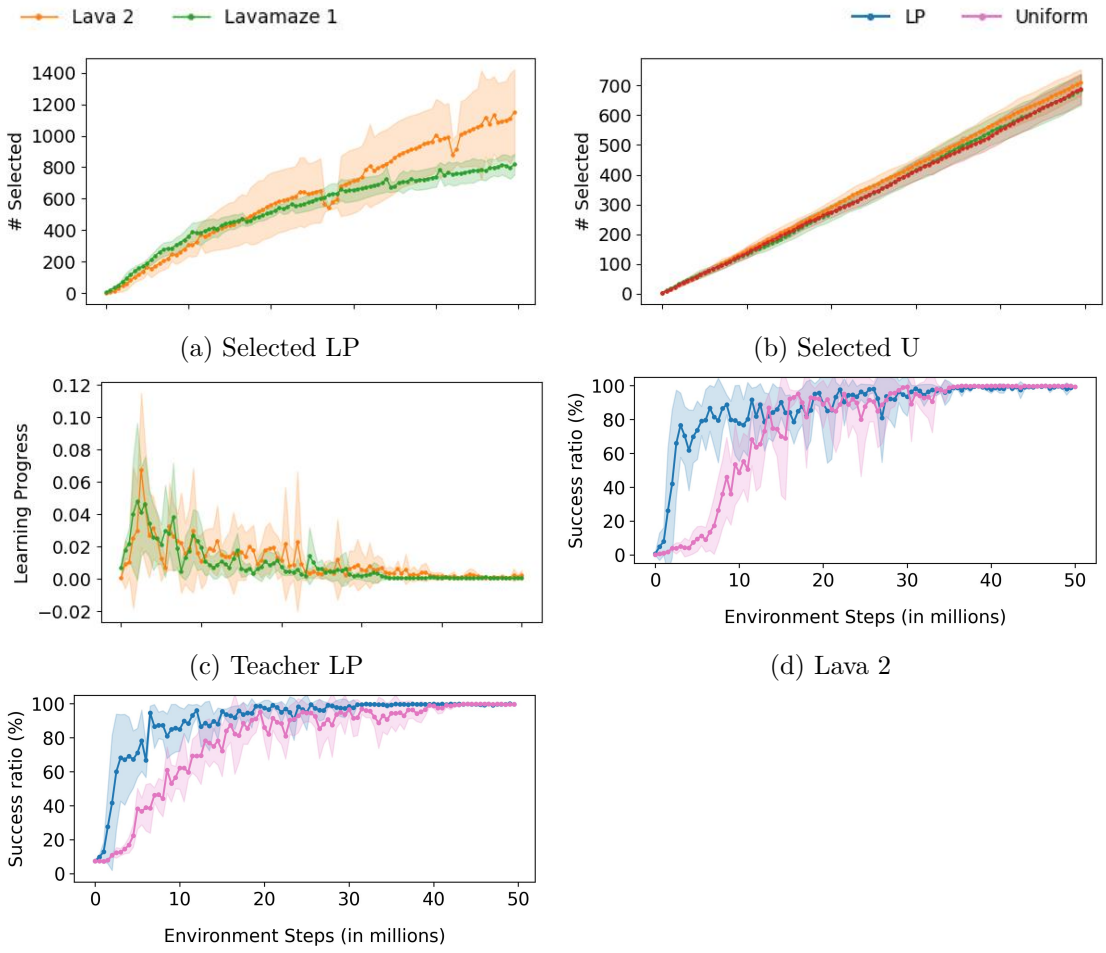
(e) Key

(f) Multiroom

Figure 4.11: Locked training

Combined

LP clearly outperforms Uniform in the Combined group, in terms of speed, performance and stability (Figure 4.14). On Combined 2, 3, and 5 it achieves high success ratio near the 100% mark in the 50,000,000 samples generated for training (Figure 4.14). In Combined 1, LP converges roughly 25,000,000 timesteps earlier (Figure 4.14d). Prioritizing environments by their current learning progress seems to benefit more difficult tasks the most. In conclusion, LP saves significant samples compared to randomly selecting environments.



(a) Selected LP

(b) Selected U

(c) Teacher LP

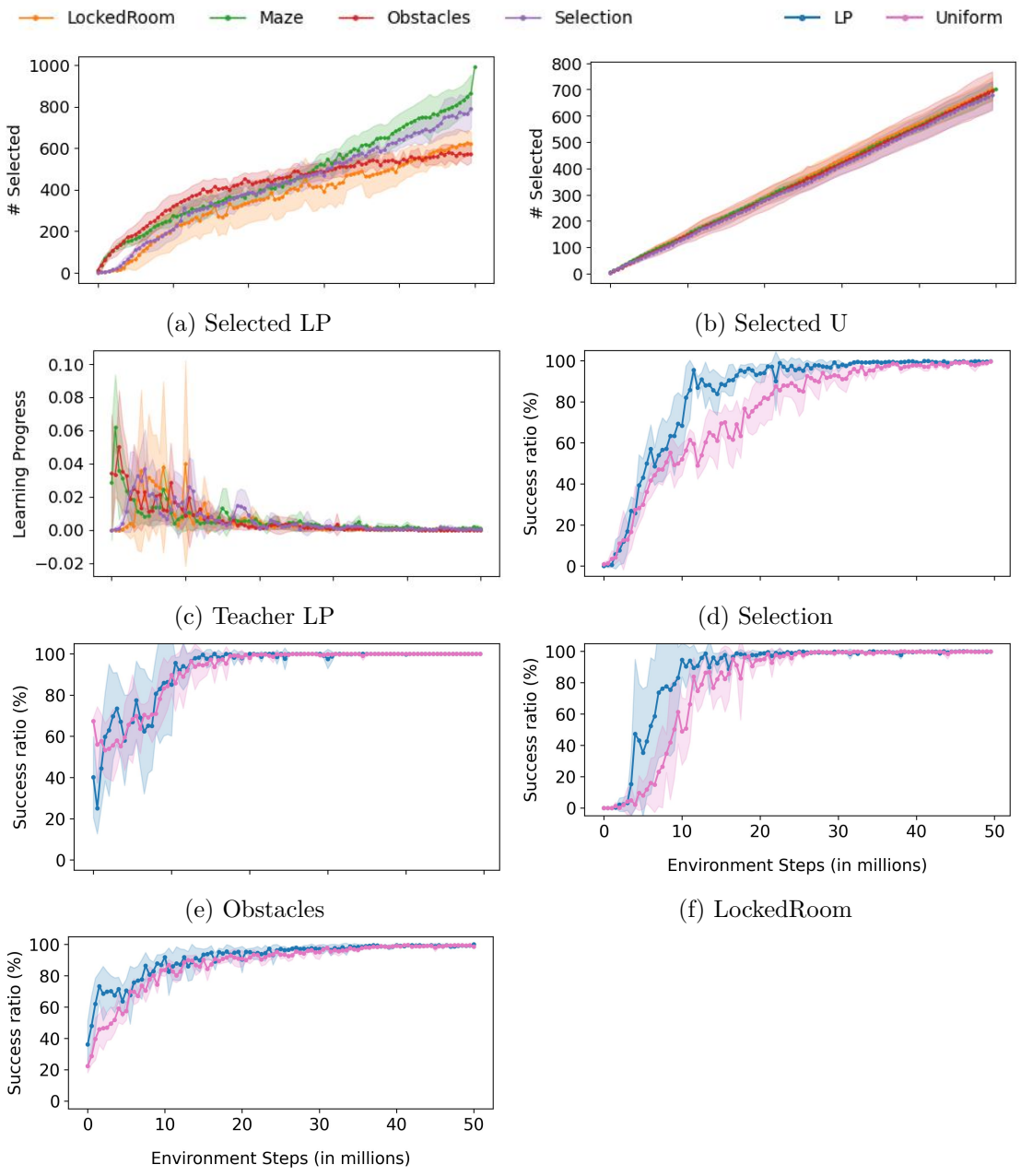
(d) Lava 2

(e) Lavamaze 1

Figure 4.12: Lava Results

4.6 Conclusion

The experiments demonstrate the benefits of applying curricula to classic reinforcement learning techniques. Even basic Uniform helps mitigate problems of sparse rewards and enhances sample efficiency. Learning Progress has proven to be a robust indicator of where training focus should lie. In smaller curricula like the ones used for Key (Figure 4.1) and LockedRoom (Figure 4.3) the teacher chooses a successful ordering of tasks, which mostly equals a curriculum if it were crafted by hand and intuition. In bigger ones (Section 4.5), the teacher becomes more important, as bigger/more difficult environments require more contiguous focus to be completed successfully (Figure 4.14). Both Uniform and LP find success in settings where Baseline, which is plain reinforcement learning, fails. LP requires significantly fewer samples than Uniform, especially as environments get more complex.



(a) Selected LP

(b) Selected U

(c) Teacher LP

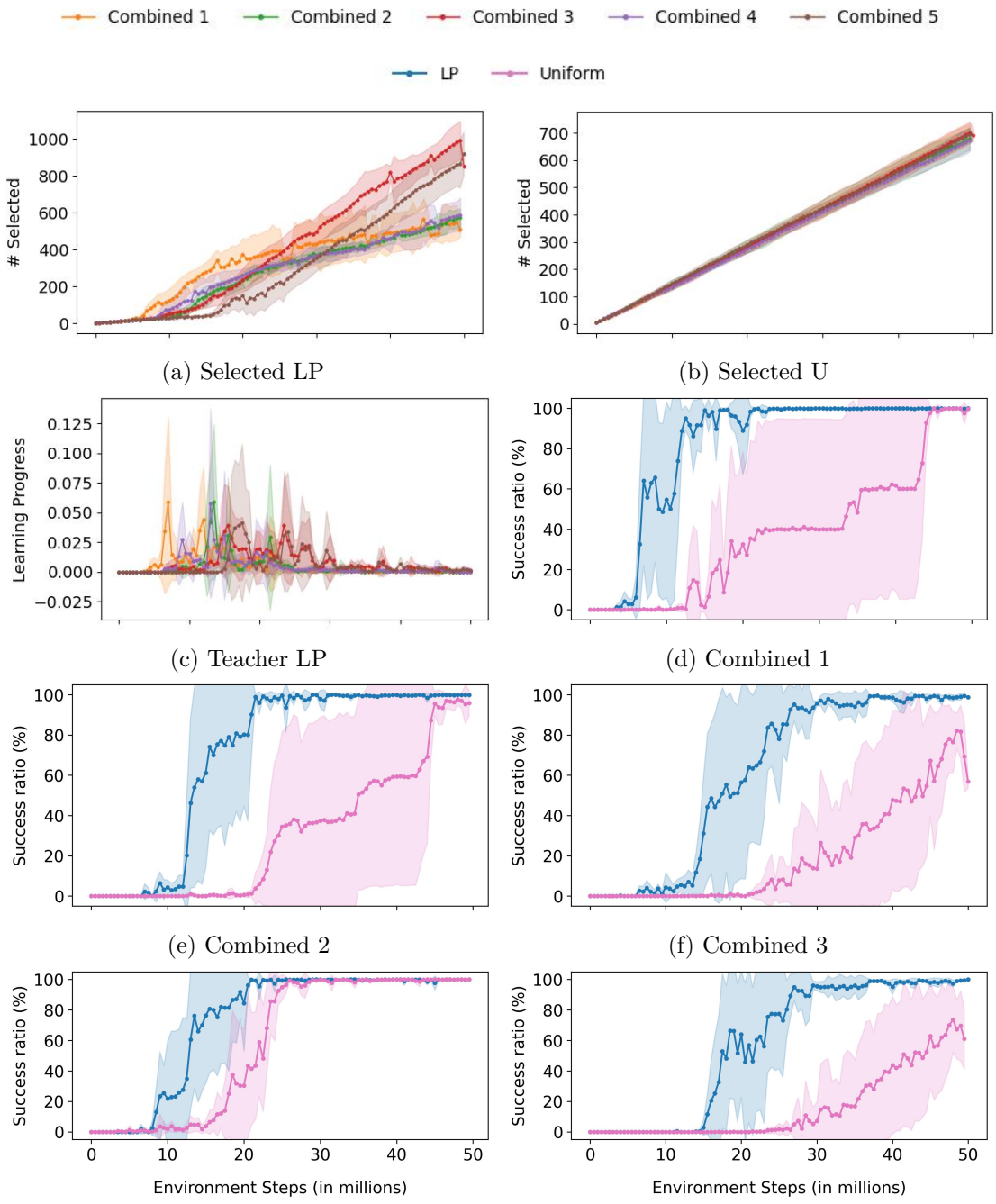
(d) Selection

(e) Obstacles

(f) LockedRoom

(g) Maze

Figure 4.13: Intermediate Results



(a) Selected LP

(b) Selected U

(c) Teacher LP

(d) Combined 1

(e) Combined 2

(f) Combined 3

(g) Combined 4

(h) Combined 5

Figure 4.14: Combined Results

CHAPTER 5

Summary and Outlook

5.1 Summary

In reinforcement learning, an agent receives implicit feedback through reward signals from the environment. Training aims to maximize the reward an agent collects. However, this process is usually sample-inefficient, heavily overfitted to a specific environment, and can even fail entirely due to sparse rewards. Partial observability further multiplies these problems, often leading to the complete failure of plain reinforcement learning algorithms in complex environments.

In this work, partial observability was successfully addressed by integrating a recurrent neural network (GRU) trained with truncated backpropagation through time. Additionally, Curriculum Learning was utilized to divide complex global environments into manageable subtasks. While these subtasks are easier to optimize, they significantly contribute to the overall optimization of the target task. This technique was combined with a teacher algorithm that monitored the agent's performance across subtasks to dynamically decide the next training environment. Specifically, the absolute change in success ratios over time served as an indicator of Learning Progress (LP), suggesting that an environment with high momentum should receive higher training focus.

The experiments, conducted in the MiniGrid, successfully demonstrate the limitations of classic reinforcement learning and highlight the substantial benefits of the applied curriculum methods:

- Overcoming Sparse Rewards: In complex tasks that require specific action sequences (such as LockedRoom or big Multiroom), the plain PPO baseline completely failed to acquire any reward. Both the Uniform and LP curricula effectively mitigated this issue, allowing the agent to robustly converge on tasks that were otherwise unsolvable.

- Significant Gains in Sample Efficiency: Even in simpler environments where standard reinforcement learning eventually succeeds, prioritizing tasks based on Learning Progress drastically reduced the required training steps. The LP teacher consistently required significantly fewer samples than both the Baseline and the Uniform sampling method by dynamically shifting focus away from already mastered or currently unlearnable tasks.
- Generalization: When applied to a bigger custom curriculum with structurally dependent tasks, the teacher managed to juggle a larger number of environments concurrently, such that the agent successfully learns a set of tasks, instead of merely a single one.

In conclusion, actively guiding an agent's training process through a Learning Progress-based curriculum not only makes training in partially observable, sparse-reward environments feasible, but also utilizes computational resources far more efficiently.

5.2 Outlook

While the inclusion of curriculum learning does provide benefits for the training of reinforcement learning agents, there are some limitations and difficulties that come with it.

The sample efficiency comes at the cost of designing the curriculum. The discrete enumeration yields benefits such as task isolation, but it comes at the cost of manually choosing and possibly crafting the environments which are used for training. Although the method is an automated curriculum, the design choices are a real bottleneck, since multiple test runs need to be conducted to assure a curriculum does not have too large gaps regarding the difficulty of each environment.

The first issue directly leads to the second issue. Increasing the number of tasks does not scale well indefinitely for two reasons. Firstly, the learning progress depends on the last observed reward in an environment E. As the number of environments grows, on average more environments get selected in between two selections of environment E, which under constantly changing network parameters might be a too outdated assessment of the agent's skill level in E. Secondly, for more resource-heavy environment simulations, there will either be a severe memory bottleneck for constantly holding all environments in memory, or a significant time overhead due to constant re-building of environments. Having only one environment with adjustable difficulty parameters (continuous task settings) does mitigate some of the described problems, but it would require far more complex world generation algorithms dependent on the difficulty parameters to get the same level of generalization.

The choice for metrics as well as their values introduces new hyperparameters to reinforcement learning. A curriculum requires a robust estimation of learning progress. Since

the episodic return is not always comparable between tasks with varying length or reward signals, different metrics can and must be tested for optimal training.

A curriculum can be an excellent bonus to add to existing methods of reinforcement learning. Future work should focus on standardizing Curriculum-Frameworks to more easily make use of these techniques.

APPENDIX A

Hyperparameters

Learning rate	$1 \cdot 10^{-4}$
Discount factor γ	0.99
GAE parameter λ	0.95
PPO epochs	5
PPO clipping parameter	0.2
Hidden dimensions	1024
Embedding dimension	512
BPTT truncation length	400
Teacher α	0.1
Teacher β	0.05
Teacher exploration ϵ	0.15

Table A.1: Optimization and Curriculum Hyperparameters

Overview of Generative AI Tools Used

In the context of this thesis, generative AI systems were used as supporting tools in the following ways:

- **Gemini:** Utilized exclusively as a proofreading and copy-editing tool to correct grammar, spelling, and sentence structure of the author's original text drafts.
- **Gemini** Used as a programming assistant to help structure, write, and debug auxiliary Python scripts (Jupyter Notebooks) for the processing and analysis of log data. Used as a LaTex syntax search engine.

The author confirms that no theoretical concepts, literature reviews, or core text passages were generated by AI. The use of AI was strictly limited to language refinement and coding assistance for data analysis. All AI-assisted outputs were thoroughly reviewed, tested, and validated by the author. The author assumes full responsibility for the complete content, accuracy, and scientific integrity of this thesis.

List of Figures

3.1 Agent-environment interaction-diagram in an MDP [9] 5
3.2 Student Teacher Framework [5] 9

4.1 Curriculum for Key 14
4.2 Key training 15
4.3 Curriculum for LockedRoom 16
4.4 LockedRoom training 17
4.5 Multiroom 18
4.6 Multiroom training 19
4.7 Lava 20
4.8 Lavamaze 20
4.9 Intermediate 21
4.10 Combined 22
4.11 Locked training 23
4.12 Lava Results 24
4.13 Intermediate Results 25
4.14 Combined Results 26

Bibliography

[1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

[2] Jeffrey L Elman. Learning and development in neural networks: The importance of starting small. *Cognition*, 48(1):71–99, 1993.

[3] Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, et al. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*, 2021.

[4] Hanna Kurniawati. Partially observable markov decision processes and robotics. *Annual Review of Control, Robotics, and Autonomous Systems*, 5(Volume 5, 2022):253–277, 2022.

[5] Tambet Matiisen, Avital Oliver, Taco Cohen, and John Schulman. Teacher–student curriculum learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9):3732–3740, 2019.

[6] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[8] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, second edition, 2018.

[10] Alan M Turing. Computing machinery and intelligence. In *Parsing the Turing Test: Philosophical and Methodological Issues in the Quest for the Thinking Computer*, pages 23–65. Springer, 2007.